



Projet n°3 : “Concevoir une application au service de la santé publique”

Soutenance de Projet
Gaillot Dimitri

=> [Lien GitHub](#)

SOMMAIRE

01

CONTEXTE

Rappel de la problématique et
présentation du jeu de données

02

NETTOYAGE

Analyse pré-exploratoire

03

ANALYSE

Argumenter et proposer des
cibles potentielles

04

CONCLUSION

01

CONTEXTE



CONTEXTE

Objectif

Trouver des idées d'applications en lien avec l'alimentation en se basant sur le jeu de données Open Food Facts



Idée d'application

Après un scan du code-barre, fournir une liste d'articles de la même catégorie / sous - catégorie présentant de meilleurs indicateurs nutritionnels.

Jeu de données

**fr.openfoodfacts.org.
products.csv**

**Fournit des informations générales et nutritionnelles sur
un panel élargi de produits alimentaires**

- **Taille** : 5,7 Go / 2 282 517 lignes x 186 colonnes
- **Valeurs manquantes** : nombreuses
 - 96 colonnes manquant 99% de données
 - 28 colonnes avec plus de 50% de données
- **Doublons** : uniquement sur une colonne de dates



02

NETTOYAGE

Nettoyage Pré-Exploratoire



Vue d'ensemble

Répartition, relations



Qualité

Données exploitables :
Manquantes, anomalies (aberrantes
/ inhabituelles)



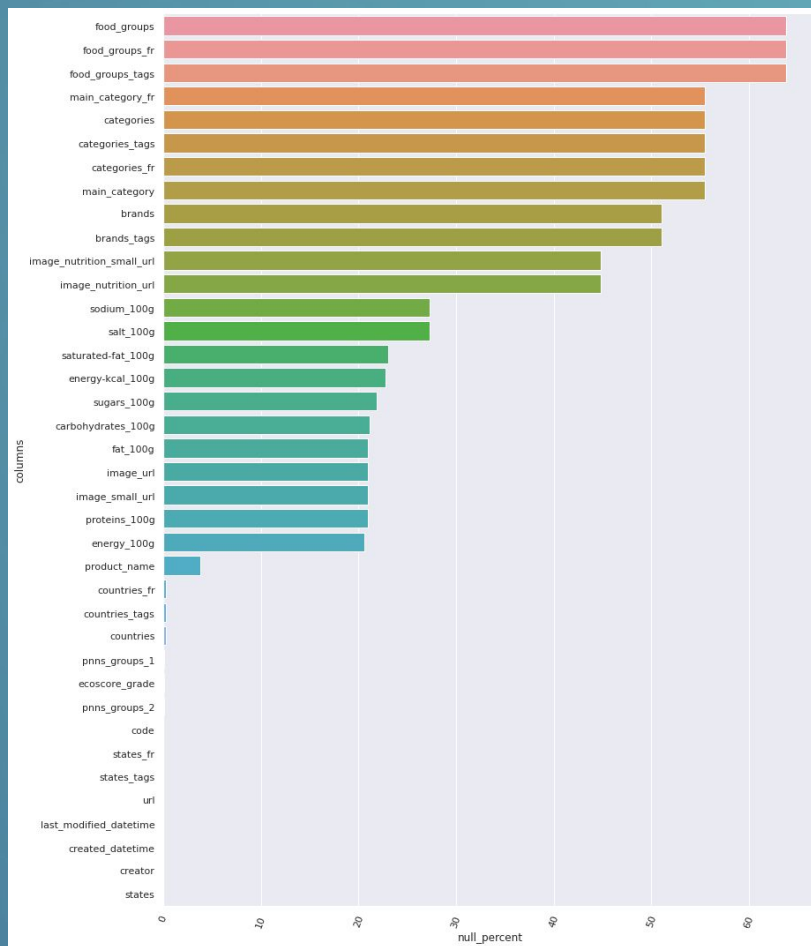
Sélection

Choix de données pertinentes,
Réduction du dataset

Vue d'ensemble

Colonnes les plus représentatives

	columns	count	null_percent
food_groups	food_groups	1455381	63.8
food_groups_fr	food_groups_fr	1455381	63.8
food_groups_tags	food_groups_tags	1455381	63.8
main_category_fr	main_category_fr	1266614	55.5
categories	categories	1266596	55.5
categories_tags	categories_tags	1266614	55.5
categories_fr	categories_fr	1266614	55.5
main_category	main_category	1266614	55.5
brands	brands	1166217	51.1
brands_tags	brands_tags	1166294	51.1
image_nutrition_small_url	image_nutrition_small_url	1023233	44.8
image_nutrition_url	image_nutrition_url	1023233	44.8
sodium_100g	sodium_100g	624058	27.3
salt_100g	salt_100g	624055	27.3
saturated-fat_100g	saturated-fat_100g	525890	23.0
energy-kcal_100g	energy-kcal_100g	519778	22.8
sugars_100g	sugars_100g	499137	21.9
carbohydrates_100g	carbohydrates_100g	480900	21.1
fat_100g	fat_100g	480453	21.0
image_url	image_url	479844	21.0
image_small_url	image_small_url	479844	21.0
proteins_100g	proteins_100g	479013	21.0
energy_100g	energy_100g	470573	20.6
product_name	product_name	86006	3.8
countries_fr	countries_fr	6391	0.3
countries_tags	countries_tags	6391	0.3
countries	countries	6386	0.3
pnns_groups_1	pnns_groups_1	1185	0.1
ecoscore_grade	ecoscore_grade	1606	0.1
pnns_groups_2	pnns_groups_2	1183	0.1
code	code	0	0.0
states_fr	states_fr	0	0.0
states_tags	states_tags	0	0.0
uri	uri	0	0.0
last_modified_datetime	last_modified_datetime	0	0.0
created_datetime	created_datetime	0	0.0
creator	creator	4	0.0
states	states	0	0.0



Sur un total de 186 colonnes :

28 colonnes contiennent 0 à 65 % de données manquantes.

96 colonnes manquent 99% de données.

Données catégorielles

```
In [36]: top_df['pnns_groups_1'].nunique()
```

```
Out[36]: 12
```

```
In [37]: top_df['pnns_groups_2'].nunique()
```

```
Out[37]: 42
```

On retrouve :

- 12 catégories
- 42 sous-catégories

```
In [40]: top_df['pnns_groups_1'].unique()
```

```
Out[40]: array(['unknown', 'Fat and sauces', 'Composite foods', 'Sugary snacks',  
               'Fruits and vegetables', 'Fish Meat Eggs', 'Beverages',  
               'Milk and dairy products', 'Cereals and potatoes', 'Salty snacks',  
               'Alcoholic beverages', nan, 'sugary-snacks'], dtype=object)
```

```
In [41]: top_df['pnns_groups_2'].unique()
```

```
Out[41]: array(['unknown', 'Dressings and sauces', 'One-dish meals',  
               'Biscuits and cakes', 'Fruits', 'Meat', 'Sweets',  
               'Sweetened beverages', 'Cheese', 'Bread', 'Fish and seafood',  
               'Salty and fatty products', 'Fruit juices', 'Dried fruits',  
               'Vegetables', 'Fats', 'Dairy desserts', 'Milk and yogurt',  
               'Pastries', 'Pizza pies and quiches', 'Legumes',  
               'Unsweetened beverages', 'Nuts', 'Cereals', 'Alcoholic beverages',  
               'Breakfast cereals', 'Appetizers', 'Processed meat',  
               'Chocolate products', 'Eggs', 'Plant-based milk substitutes',  
               'Sandwiches', 'Ice cream', 'Soups',  
               'Teas and herbal teas and coffees', 'Potatoes',  
               'Artificially sweetened beverages', 'Waters and flavored waters',  
               nan, 'Offals', 'Fruit nectars', 'pastries',  
               'Pizza pies and quiche'], dtype=object)
```

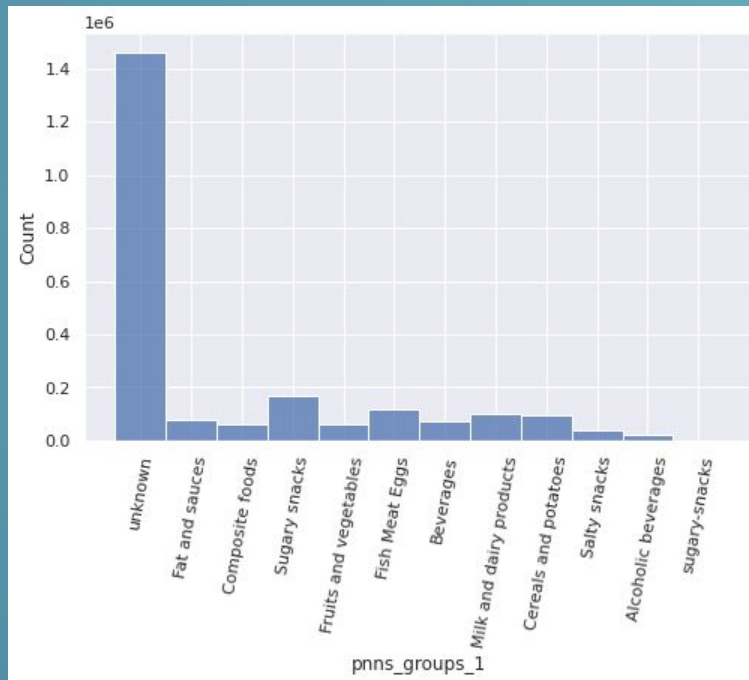
Données catégorielles

```
top_df['pnns_groups_1'].value_counts()
```

unknown	1461312
Sugary snacks	169932
Fish Meat Eggs	115729
Milk and dairy products	102703
Cereals and potatoes	95597
Fat and sauces	79141
Beverages	75157
Fruits and vegetables	62980
Composite foods	60640
Salty snacks	37917
Alcoholic beverages	20222
sugary-snacks	2

Name: pnns_groups_1, dtype: int64

Une majorité de valeurs avec un label “unknown”.



Sélection de features principales

```
In [49]: sm_df = top_df.loc[:,['product_name', 'pnns_groups_1', 'pnns_groups_2', 'code', 'energy-kcal_100g', 'energy_100g',
    'fat_100g', 'saturated-fat_100g', 'carbohydrates_100g', 'sugars_100g',
    'proteins_100g', 'salt_100g', 'sodium_100g']]
```

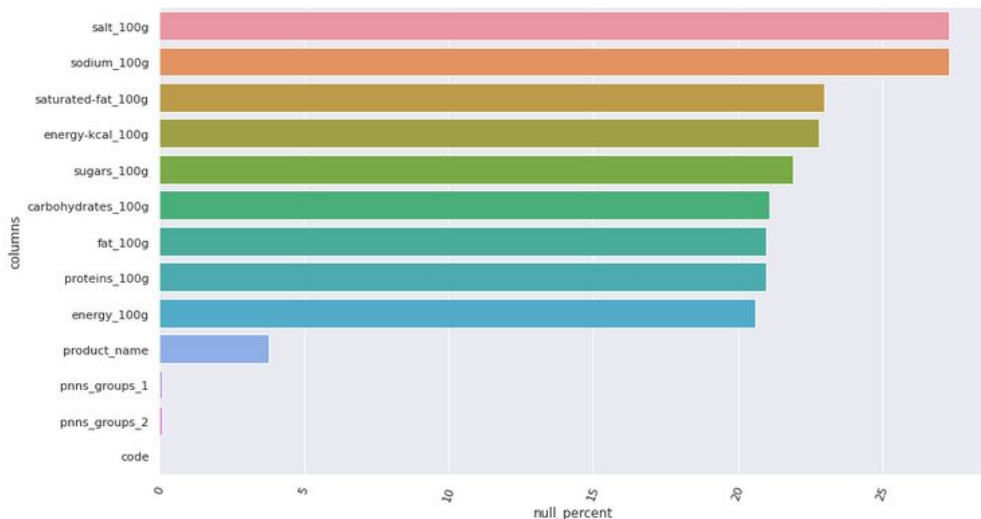
```
In [50]: df.shape
```

```
Out[50]: (2282517, 184)
```

```
In [51]: sm_df.shape
```

```
Out[51]: (2282517, 13)
```

```
In [52]: top_sm_df = get_df_columns_na(sm_df).sort_values(by='null_percent',ascending=False)
plot_df_xy(top_sm_df.reset_index(), 'null_percent', 'columns', 7)
```



Sélection des features répondant le mieux aux besoins de l'application :

- Données disponibles
- Code-barres & catégories
- Macro-nutriments (éléments majoritaires des aliments)

	columns	count	null_percent
	salt_100g	624055	27.3
	sodium_100g	624058	27.3
	saturated-fat_100g	525890	23.0
	energy-kcal_100g	519778	22.8
	sugars_100g	499137	21.9
	carbohydrates_100g	480900	21.1
	fat_100g	480453	21.0
	proteins_100g	479013	21.0
	energy_100g	470573	20.6
	product_name	86006	3.8
	pnns_groups_1	1185	0.1
	pnns_groups_2	1183	0.1
	code	0	0.0

Gestion de données aberrantes

Retrait d'aliments présentant des données aberrantes :

- Valeurs par 100g supérieures à 100g
- Somme des valeurs par 100g par aliment supérieures à 100g
- Valeurs énergétiques irréalistes (plus de 900 kCal / 100g, au delà du maximum possible avec 100g de graisses)



Gestion de données manquantes

Utilisation de SimpleImputer de SKLearn avec une stratégie "Médiane".

Données manquantes :

- 4.6% de noms de produits
- 0.1% de catégories

=> retrait / drop des lignes concernées, résultat :

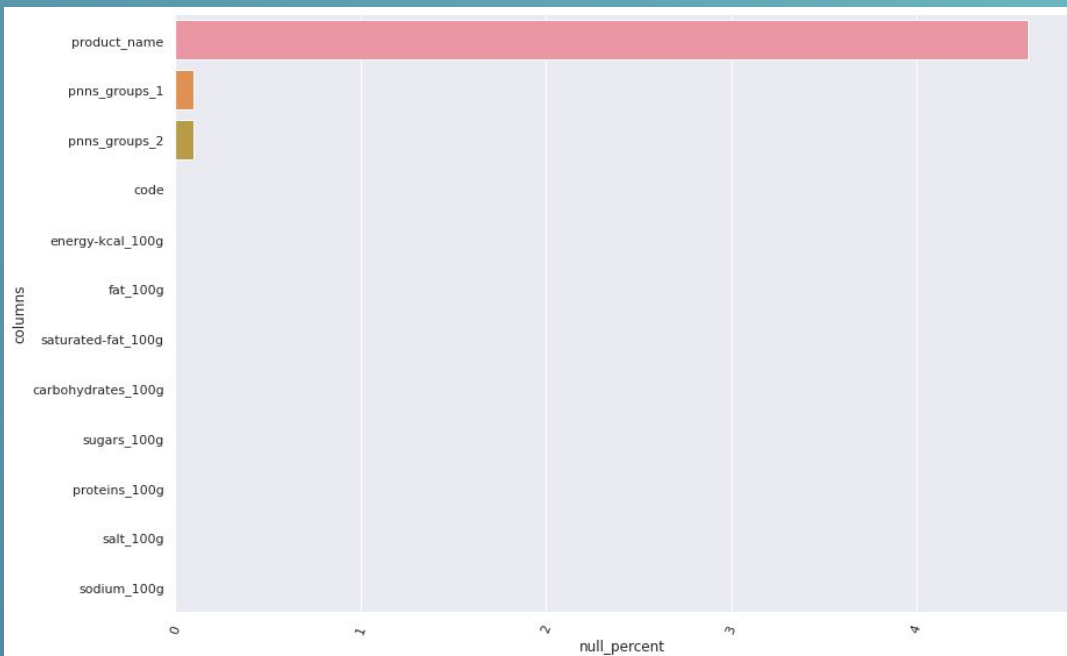
- 1 805 569 lignes
- 12 colonnes

	columns	count	null_percent
product_name	product_name	83937	4.6
pnns_groups_1	pnns_groups_1	1185	0.1
pnns_groups_2	pnns_groups_2	1183	0.1
code	code	0	0.0
energy-kcal_100g	energy-kcal_100g	0	0.0
fat_100g	fat_100g	0	0.0
saturated-fat_100g	saturated-fat_100g	0	0.0
carbohydrates_100g	carbohydrates_100g	0	0.0
sugars_100g	sugars_100g	0	0.0
proteins_100g	proteins_100g	0	0.0
salt_100g	salt_100g	0	0.0
sodium_100g	sodium_100g	0	0.0

```
In [77]: import numpy as np
         from sklearn.impute import SimpleImputer

         all_num_cols = energy_cols + sm_num_cols

         imp_med = SimpleImputer(missing_values=np.nan, strategy='median')
         sm_df[all_num_cols] = imp_med.fit_transform(sm_df[all_num_cols])
```



Gestion de données manquantes

Produits :

Il reste 83 937 produits (4.6%)
ayant pour nom “NaN” enregistré en valeur texte.

=> Retrait de ces dernières.

Codes-barre :

La description du dataset précise que les
codes-barre avec un préfixe “200” correspondent
à des données manquantes.

=> retrait des lignes correspondantes, l’absence
de code-barre rendant les lignes inutilisables pour
l’application choisie initialement.

Décompte données nulles après traitement :

	columns	count	null_percent
product_name	product_name	0	0.0
pnnns_groups_1	pnnns_groups_1	0	0.0
pnnns_groups_2	pnnns_groups_2	0	0.0
code	code	0	0.0
energy-kcal_100g	energy-kcal_100g	0	0.0
fat_100g	fat_100g	0	0.0
saturated-fat_100g	saturated-fat_100g	0	0.0
carbohydrates_100g	carbohydrates_100g	0	0.0
sugars_100g	sugars_100g	0	0.0
proteins_100g	proteins_100g	0	0.0
salt_100g	salt_100g	0	0.0
sodium_100g	sodium_100g	0	0.0

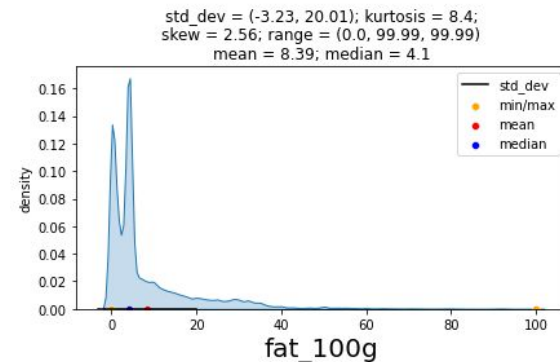
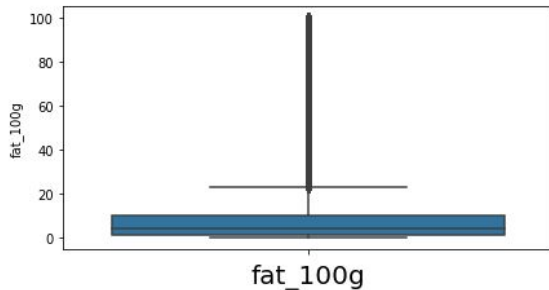
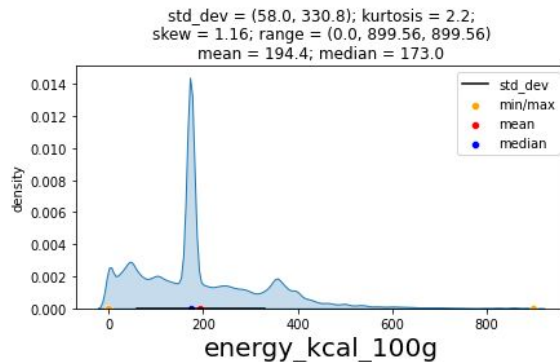
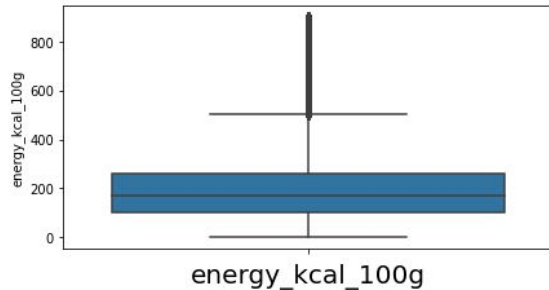
Au final :

- 1 708 030 lignes (74.8% volume initial)
- 12 colonnes (6.5% volume initial)



03

ANALYSE



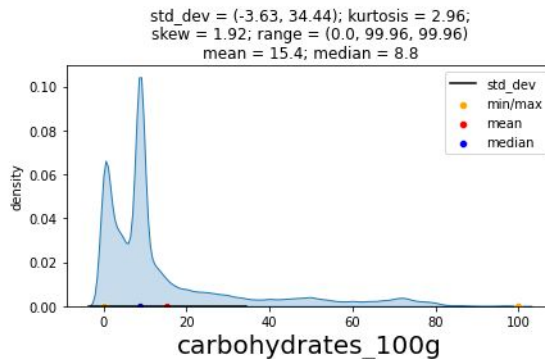
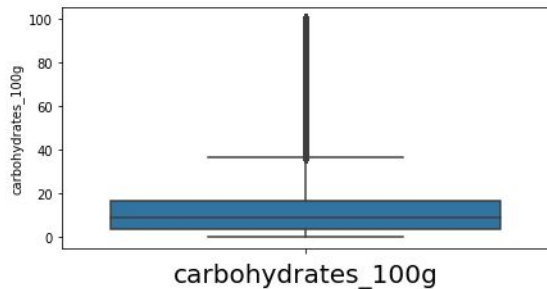
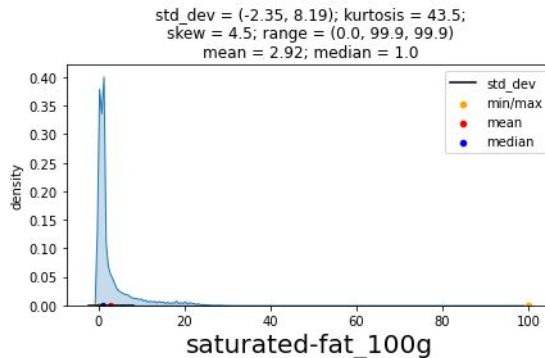
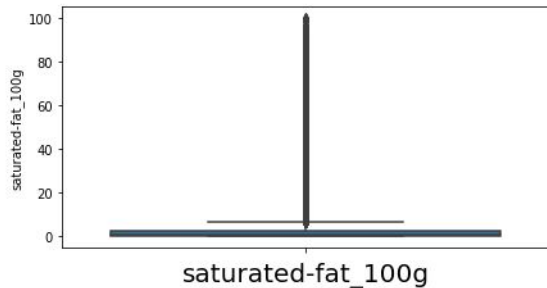
Std_dev : déviation standard, mesure de dispersion / variation de la distribution

Kurtosis : coefficient d'aplatissement

Skewness : coefficient d'asymétrie

Mean : moyenne

Median : médiane



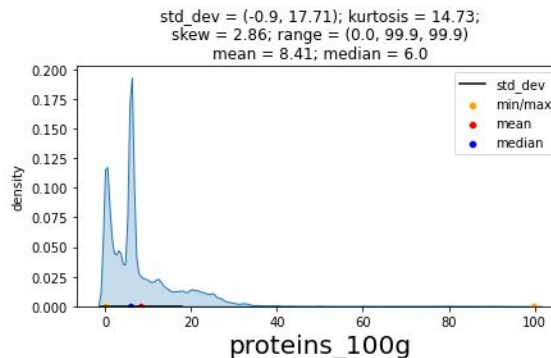
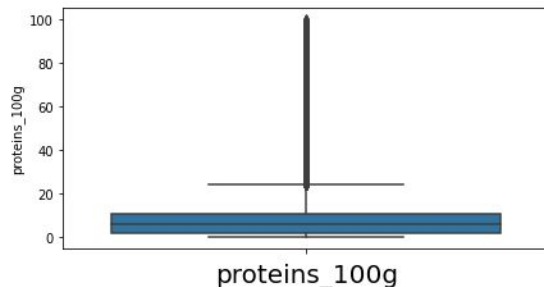
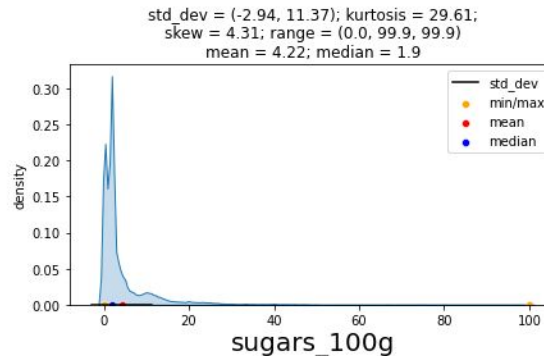
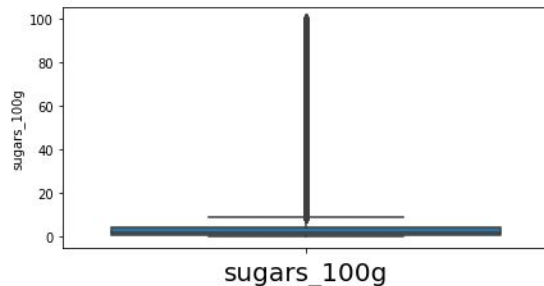
Std_dev : déviation standard, mesure de dispersion / variation de la distribution

Kurtosis : coefficient d'aplatissement

Skewness : coefficient d'asymétrie

Mean : moyenne

Median : médiane



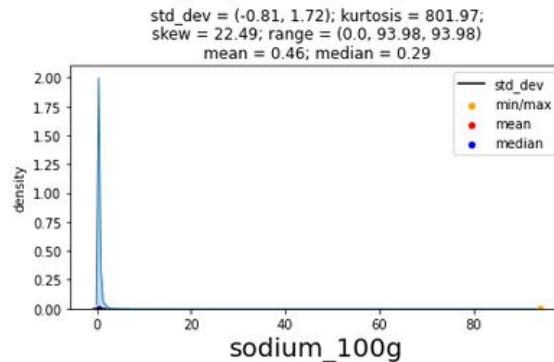
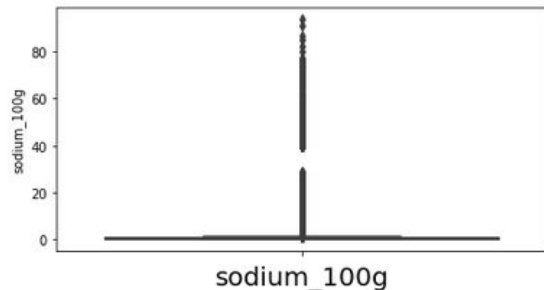
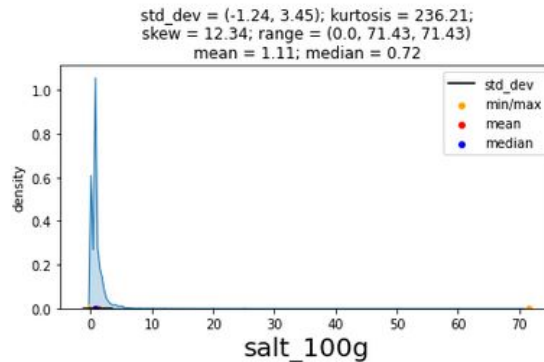
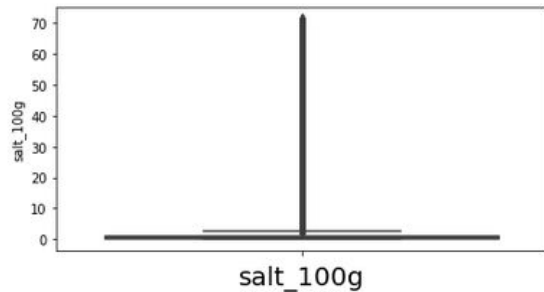
Std_dev : déviation standard, mesure de dispersion / variation de la distribution

Kurtosis : coefficient d'aplatissement

Skewness : coefficient d'asymétrie

Mean : moyenne

Median : médiane



Std_dev : déviation standard, mesure de dispersion / variation de la distribution

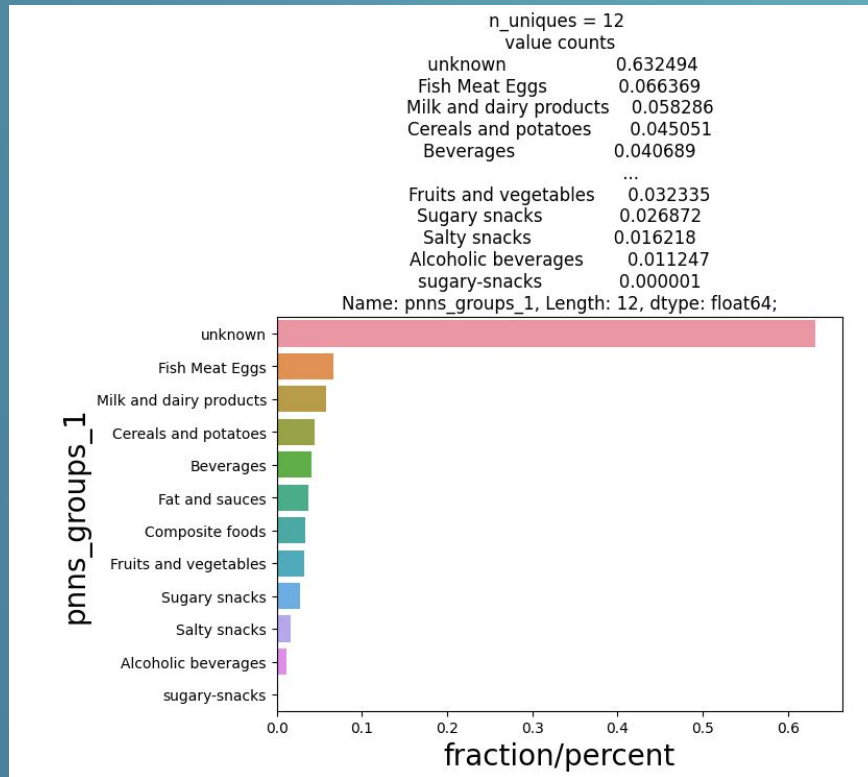
Kurtosis : coefficient d'aplatissement

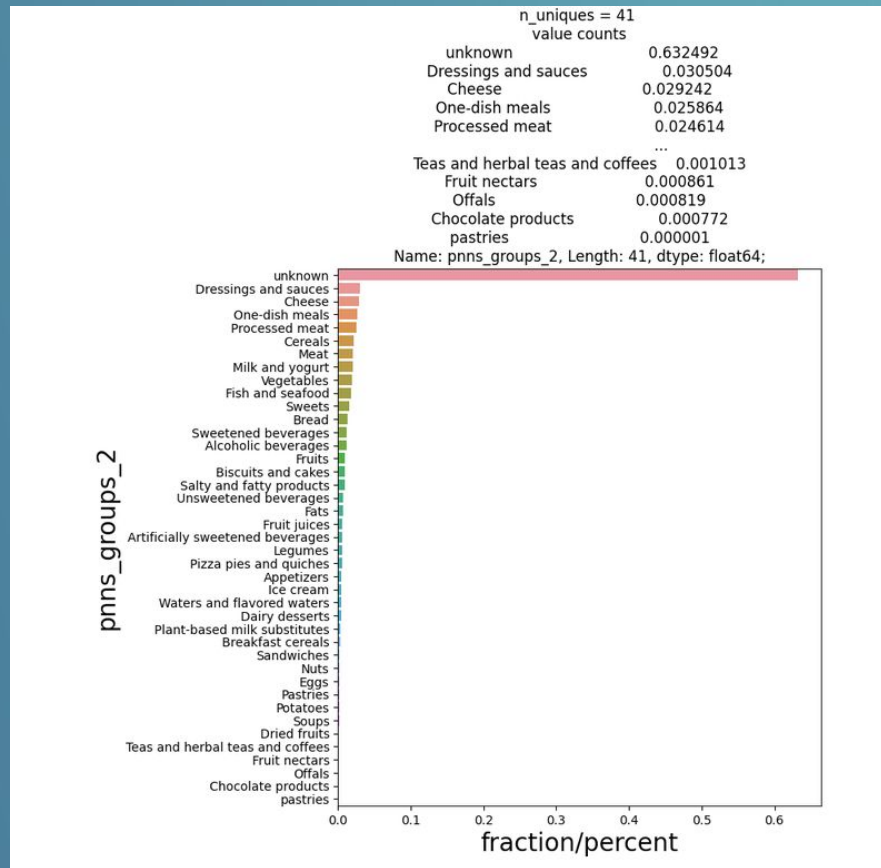
Skewness : coefficient d'asymétrie

Mean : moyenne

Median : médiane

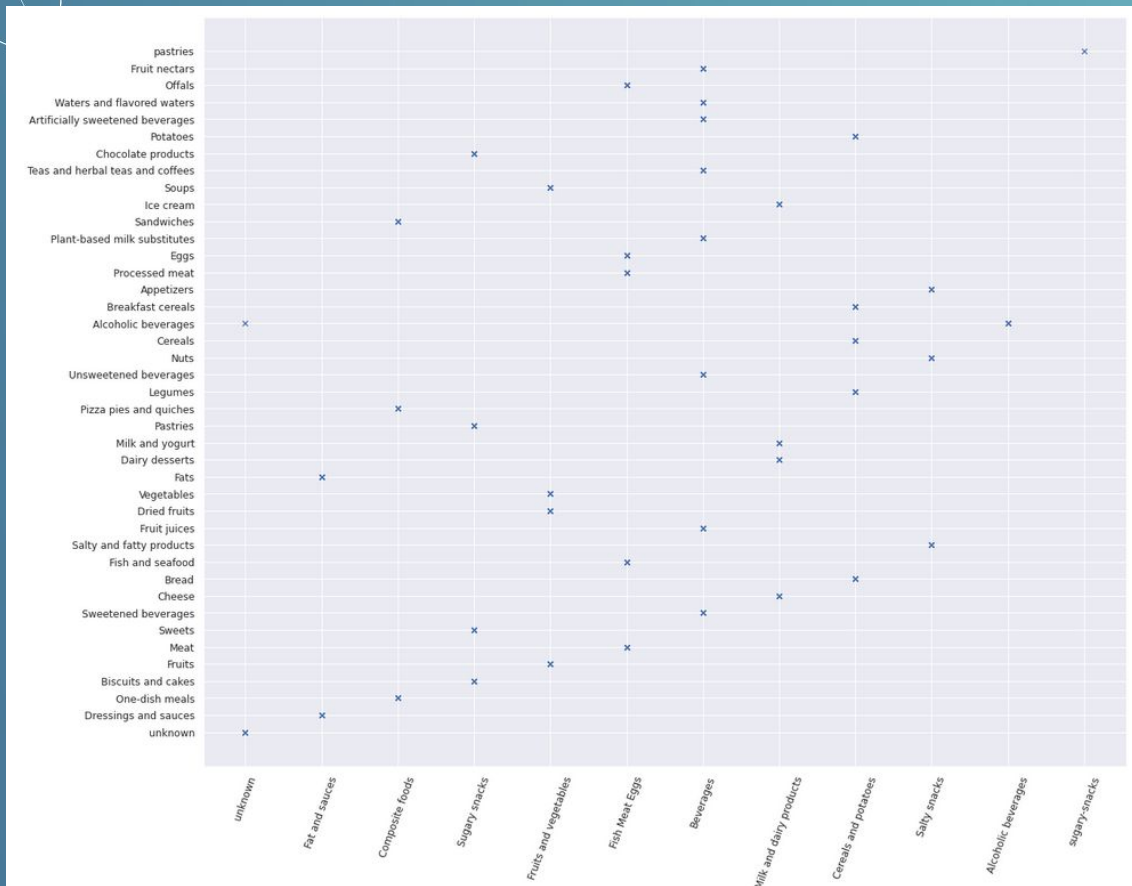
Variables catégorielles 1/2





Analyse bi-variée

Variables cat. vs cat.

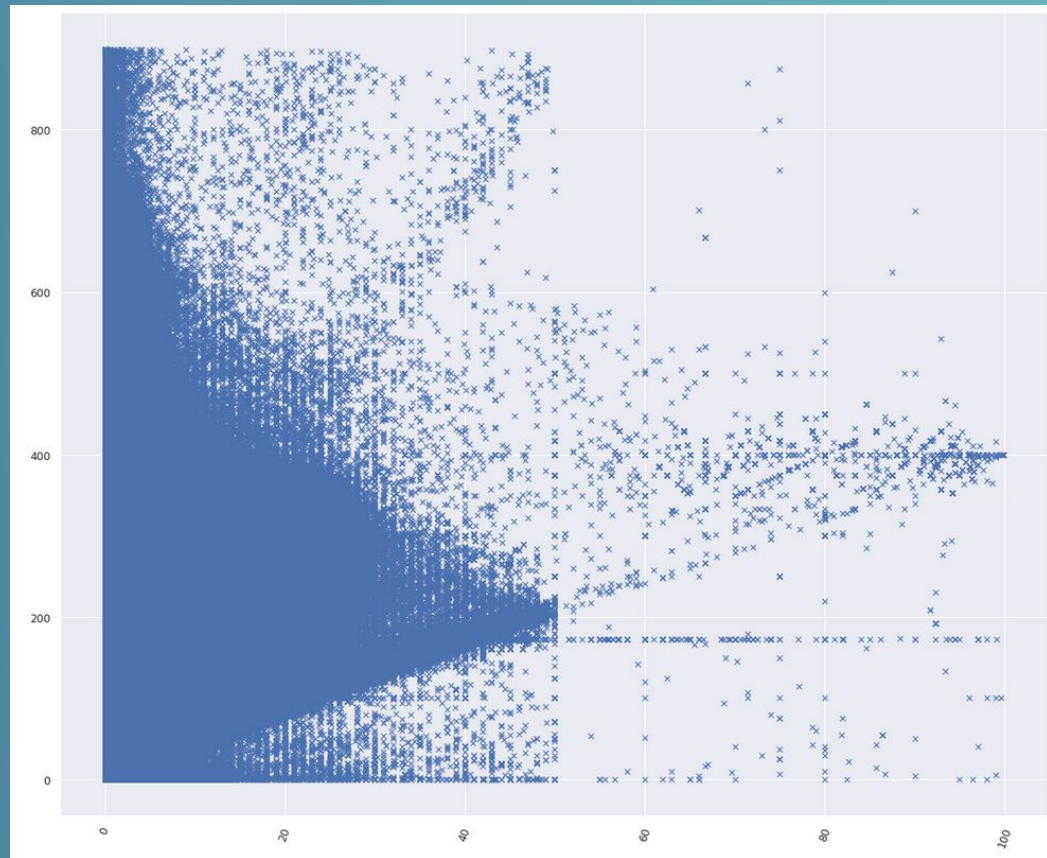


Analyse bi-variée

Variables catégorielles 2/2

Sucres vs Energie

```
: # num / num  
  
plt.figure(figsize=(18, 15), dpi=80)  
plt.xticks(rotation=70)  
x = df['sugars_100g']  
y = df['energy_kcal_100g']  
plt.plot(x,y, 'x')  
plt.show()  
# titres  
#taille
```

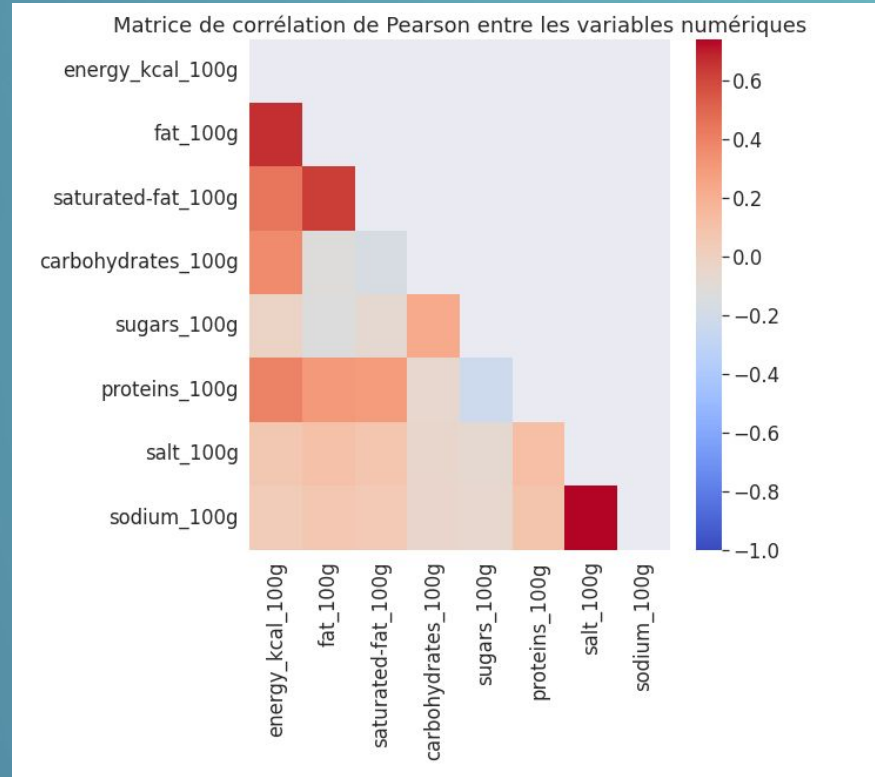


Analyse multi-variée

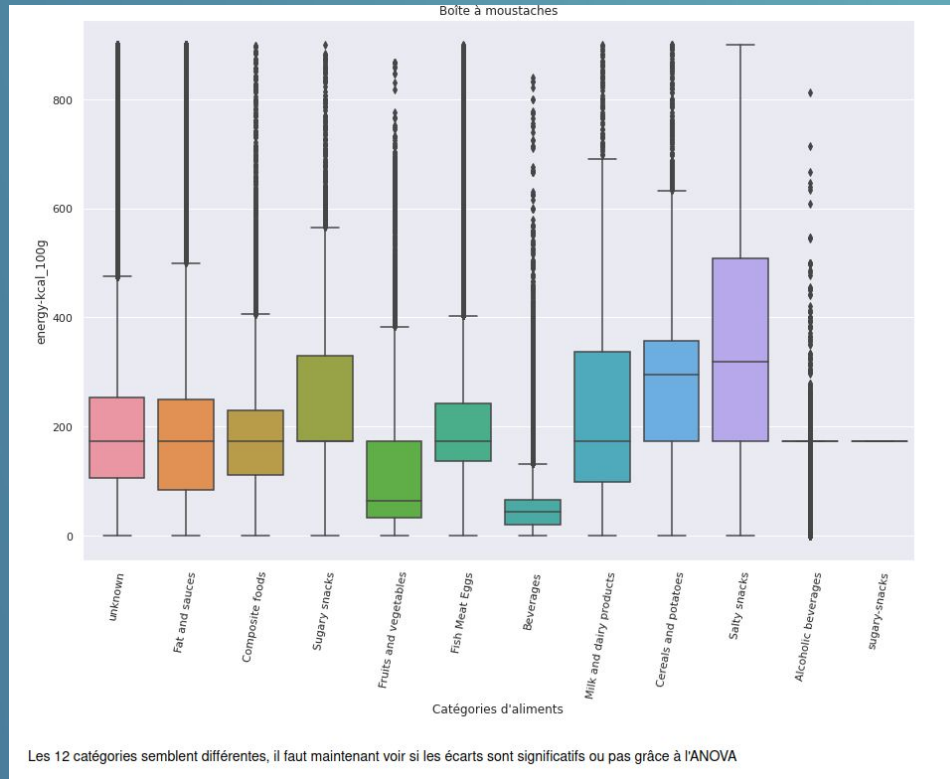
On peut retirer du tableau les informations suivantes :

- **fat_100g** et **saturated-fat_100g** sont fortement corrélés
- **energy-kcal_100g** : forte corrélation avec :
 - **Fat_100g**
 - **Saturated-fat_100g**
 - **Carbohydrates_100g**
 - **Proteins_100g**
- **sodium_100g** et **salt_100g** sont fortement corrélés
- **sugars_100g** sont corrélés avec **carbohydrates_100g**

Matrice de corrélation



Analyse de la variance (ANOVA)



Résumé PCA

=====
Total: 10 composants

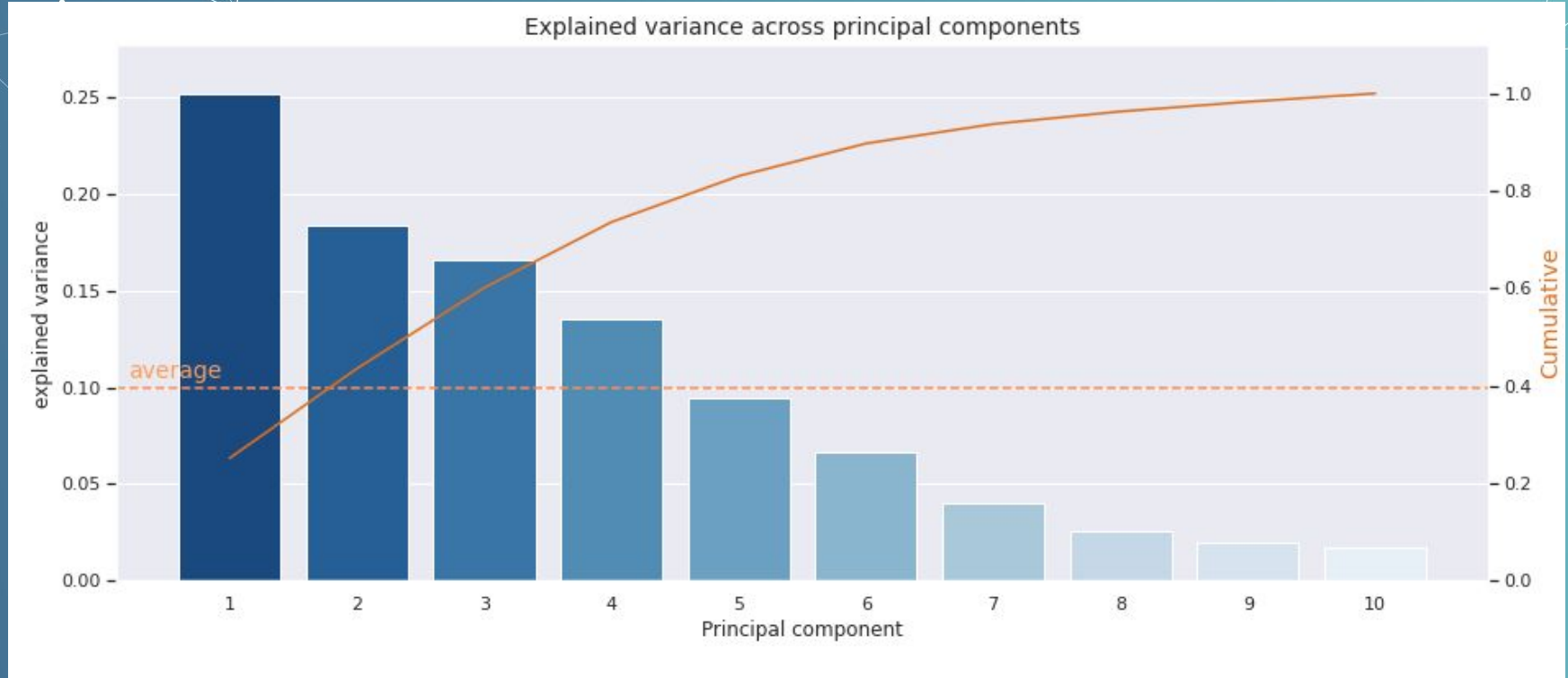
Moyenne de la variance expliquée : 0.1

	explained variance	cumulative
1	0.251719	0.251719
2	0.183806	0.435525
3	0.165769	0.601293
4	0.134887	0.736180
5	0.094456	0.830636
6	0.066787	0.897423
7	0.039894	0.937317
8	0.025744	0.963061
9	0.019960	0.983021
10	0.016979	1.000000

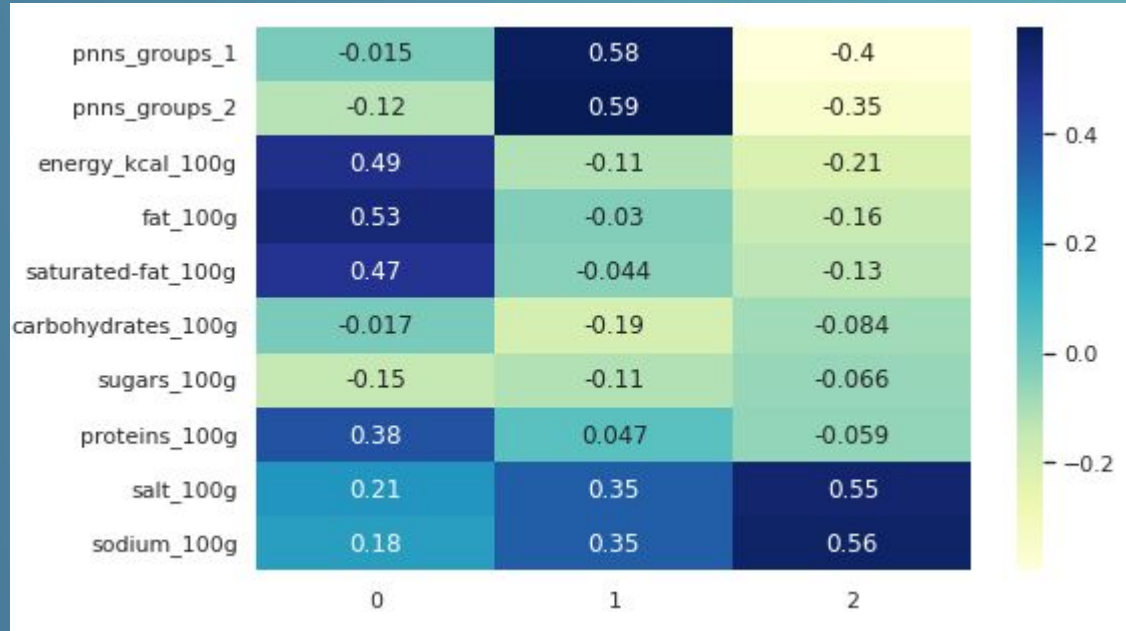
Le 1er composant représente 25% de la variance.

Si l'on veut réduire les dimensions existantes et retenir 93% de la variance originale, on peut rester sur 7 composants.

Principal Components Analysis



Principal Components Analysis - Factor Loading Matrix -





04

CONCLUSION

Conclusion

Pertinence des données :

Les données initiales sont intéressantes mais requièrent beaucoup de nettoyage et transformations avant d'être utilisables dans le cadre de l'application choisie initialement.

7 composants représentent 93% de la variance originale, mais il peut être intéressant de préserver l'ensemble des données sélectionnées pour préserver une précision maximale.

Abstract geometric line art in the top corners of the slide, consisting of interconnected lines and dots forming various polygonal shapes.

Merci

Pour votre attention.

GAILLOT Dimitri